COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	**CPE212 / Automating Server Management**	**SECTION:**	**CPE31S2**
DATE PERFORMED:	**Aug 24 2026**	**DATE SUBMITTED:**	**Aug 24 2026**
NAME:	**Kenzo Lim**	**INSTRUCTOR:**	**Engr. Jimlord Quejado**

ACTIVITY NO. 4 **Running Elevated Ad hoc Commands**

1. PROCEDURE OUTPUT

Issue the following command:
ansible all -m apt -a update_cache=true

```
kenzo@workstation:~/ansible_proj$ ansible all -m apt -a update_cache=true
[WARNING]: Updating cache and auto-installing missing dependency: python3-apt
[WARNING]: Platform linux on host vm01 is using the discovered Python interpreter at
/usr/bin/python3.9, but future installation of another Python interpreter could change the
meaning of that path. See https://docs.ansible.com/ansible-
core/2.17/reference_appendices/interpreter_discovery.html for more information.
vm01 | FAILED! => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.9"
    },
    "changed": false,
    "cmd": "update",
    "msg": "[Errno 2] No such file or directory: b'update'",
    "rc": 2,
    "stderr": "",
    "stderr_lines": [],
    "stdout": "",
    "stdout_lines": []
}
```

Issue the command ansible
all -m apt -a update_cache=true --become --ask-become-pass.

```
kenzo@workstation:~/ansible_proj$ ansible all -m apt -a update_cache=true --become --ask-become-p
ass
BECOME password:
[WARNING]: Updating cache and auto-installing missing dependency: python3-apt
[WARNING]: Platform linux on host vm01 is using the discovered Python interpreter at
/usr/bin/python3.9, but future installation of another Python interpreter could change the
meaning of that path. See https://docs.ansible.com/ansible-
core/2.17/reference_appendices/interpreter_discovery.html for more information.
vm01 | FAILED! => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.9"
    },
    "changed": false,
    "cmd": "update",
    "msg": "[Errno 2] No such file or directory: b'update'",
    "rc": 2,
    "stderr": "",
    "stderr_lines": [],
    "stdout": "",
    "stdout_lines": []
}
```

(These error I encountered was only because of the packaged used, I am using centos as the server in my

personal computer)

Corrected version:

```
kenzo@workstation:~/ansible_proj$ ansible all -m dnf -a update_cache=true
[WARNING]: Platform linux on host vm01 is using the discovered Python interpreter at
/usr/bin/python3.9, but future installation of another Python interpreter could change the
meaning of that path. See https://docs.ansible.com/ansible-
core/2.17/reference_appendices/interpreter_discovery.html for more information.
vm01 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.9"
    },
    "changed": false,
    "msg": "Cache updated",
    "rc": 0,
    "results": []
}
```

```
kenzo@workstation:~/ansible_proj$ ansible all -m dnf -a update_cache=true --become --ask-become-p
ass
BECOME password:
[WARNING]: Platform linux on host vm01 is using the discovered Python interpreter at
/usr/bin/python3.9, but future installation of another Python interpreter could change the
meaning of that path. See https://docs.ansible.com/ansible-
core/2.17/reference_appendices/interpreter_discovery.html for more information.
vm01 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.9"
    },
    "changed": false,
    "msg": "Cache updated",
    "rc": 0,
    "results": []
}
```

Let's try to install VIM, which is an almost compatible version of the UNIX editor Vi. To do this, we will just changed the module part in 1.1 instruction.

Here is the command: **ansible all -m apt -a name=vim-nox --become --ask-become-pass.**
The command would take some time after typing the password because the
local machine instructed the remote servers to actually install the package.

```
kenzo@workstation:~/ansible_proj$ ansible all -m dnf -a name=vim-nox --become --ask-become-pass
BECOME password:
[WARNING]: Platform linux on host vm01 is using the discovered Python interpreter at
/usr/bin/python3.9, but future installation of another Python interpreter could change the
meaning of that path. See https://docs.ansible.com/ansible-
core/2.17/reference_appendices/interpreter_discovery.html for more information.
vm01 | FAILED! => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.9"
    },
    "changed": false,
    "failures": [
        "No package vim-nox available."
    ],
    "msg": "Failed to install some of the specified packages",
    "rc": 1,
    "results": []
}
kenzo@workstation:~/ansible_proj$ ansible all -m dnf -a name=vim-nox --become --ask-become-pass
```

(again I encountered an error, but this time vim-nox is a debian/ubuntu package, I corrected it by replacing vim-nox with vim-enhanced)

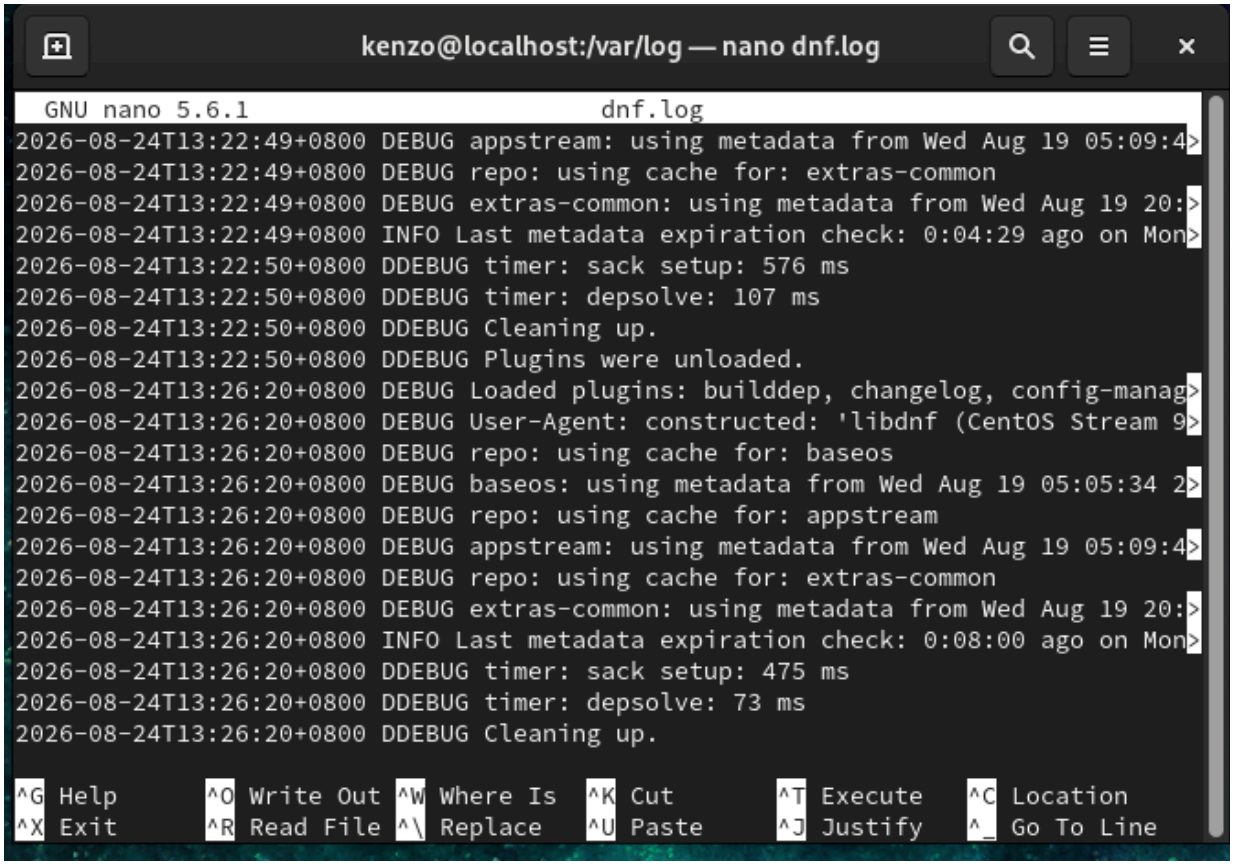
```
kenzo@workstation:~/ansible_proj$ ansible all -m dnf -a name=vim-enhanced --become --ask-become-p
ass
BECOME password:
[WARNING]: Platform linux on host vm01 is using the discovered Python interpreter at
/usr/bin/python3.9, but future installation of another Python interpreter could change the
meaning of that path. See https://docs.ansible.com/ansible-
core/2.17/reference_appendices/interpreter_discovery.html for more information.
vm01 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.9"
    },
    "changed": false,
    "msg": "Nothing to do",
    "rc": 0,
    "results": []
}
```

a. Verify that you have installed the package in the remote servers. Issue the command which vim and the command apt search vim-nox respectively. Was the command successful?

```
[kenzo@localhost ~]$ which vim
/usr/bin/vim
[kenzo@localhost ~]$ dnf search vim-enhanced
Last metadata expiration check: 0:07:45 ago on Mon 24 Aug 2026 13:19:19 PST.
====================== Name Exactly Matched: vim-enhanced ======================
vim-enhanced.x86_64 : A version of the VIM editor which includes recent
                    : enhancements
[kenzo@localhost ~]$
```

It was successful because after typing which vim, it produced an output of /usr/bin/vim which is the location of the installed vim

b. Check the logs in the servers using the following commands: cd /var/log. After this, issue the command ls, go to the folder apt and open history.log. Describe what you see in the history.log.

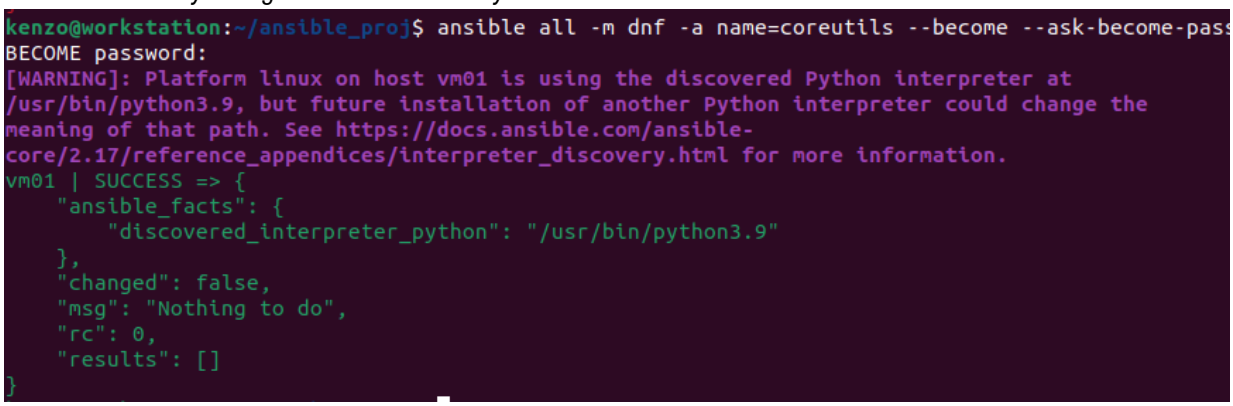


The logs show debug information, some are repository and cache activities which indicates that system is checking metadata for repositories and using existing cache to speed up the process

This time, we will install a package called snapd. Snap is pre-installed in Ubuntu system. However, our goal is to create a command that checks for the latest installation package.

a. Issue the command: ansible all -m apt -a name=snapd --become --ask-become-pass
Can you describe the result of this command? Is it a success? Did it change anything in the remote servers?

It did not make any change since it was already installed



(In this part I used coreutils as the package installed)

b. Now, try to issue this command: **ansible all -m apt -a "name=snapd state=latest" --become --ask-become-pass**

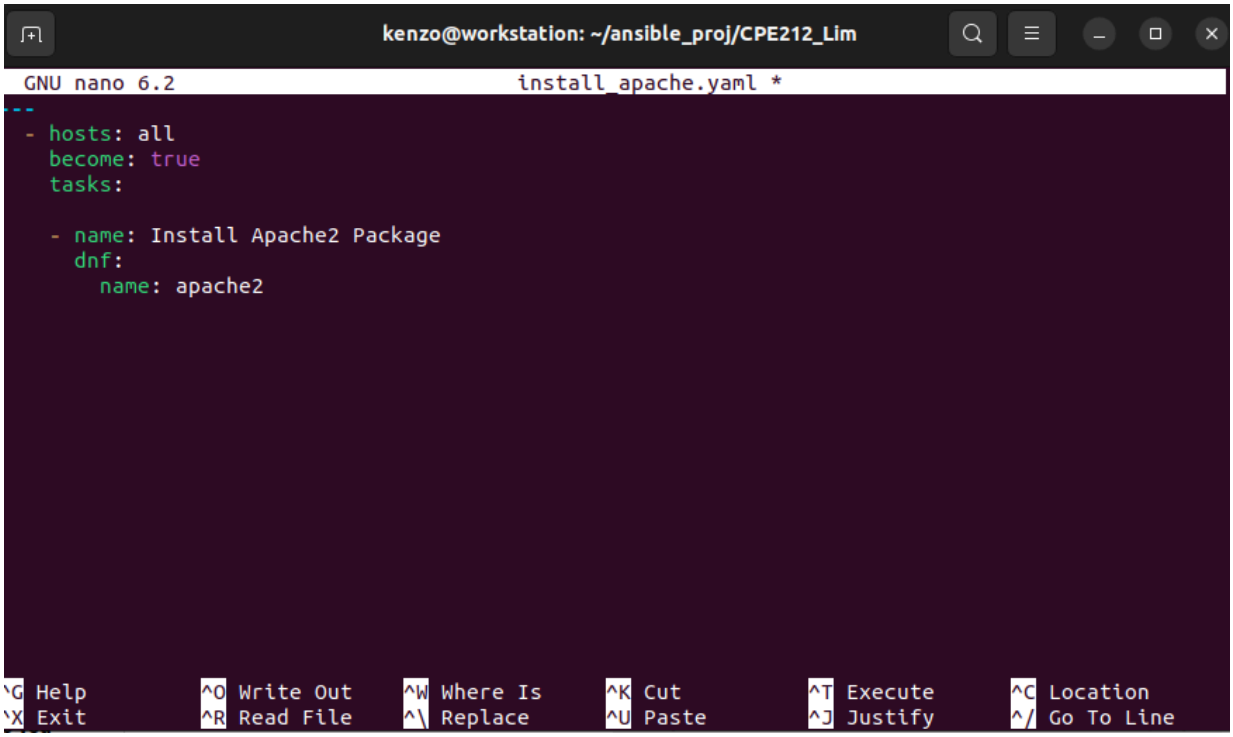
```
kenzo@workstation:~/ansible_proj$ ansible all -m dnf -a "name=coreutils state=latest" --become --
ask-become-pass
BECOME password:
[WARNING]: Platform linux on host vm01 is using the discovered Python interpreter at
/usr/bin/python3.9, but future installation of another Python interpreter could change the
meaning of that path. See https://docs.ansible.com/ansible-
core/2.17/reference_appendices/interpreter_discovery.html for more information.
vm01 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.9"
    },
    "changed": false,
    "msg": "Nothing to do",
    "rc": 0,
    "results": []
}
kenzo@workstation:~/ansible_proj$
```

Describe the output of this command. Notice how we added the command state=latest and placed them in double quotations.

With ad hoc commands, we can simplify the administration of remote servers. For example, we can install updates, packages, and applications, etc. However, the real strength of ansible comes from its playbooks. When we write a playbook, we can define the state that we want our servers to be in and the place or commands that ansible will carry out to bring to that state. You can use an editor to create a playbook. Before we proceed, make sure that you are in the directory of the repository that we use in the previous activities (CPE232_yourname). Issue the command nano install_apache.yml. This will create a playbook file called install_apache.yaml. The .yaml is the basic standard extension for playbook files.

```
kenzo@workstation:~/ansible_proj$ cat ~/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAACAQDyS3ApvUDPYVVAxt4uF6RWQVA5/krqpWYD752rMXWMjExbDDU2Qjso/LVPu
e8nN5iuraFyU2ngaDhFjYL4sEDi/ZnWkjacIBulPRjvYTaKzklh1Ib1bTVos/AdbjH2J2wjsA5sX/IMOLs4ct1pShWvaNMpMm
o39TiKFSySocr6O6iCpc7UnN1fAdmmBs1r3hDnN/DOrfK9bErXBxyDUgAp1s4fTaUh0lhI2cUo+C3KtJHkeyrXcT/PqF9Bmzj
0kj1xLqEcV95oG8S8wvcYd1qWe0FxxCBE2z+yJ1qfiNICc200Jhm1EbKUn0OVCSVFeh8Lwn/ZFwYg2nG2IwzVbD3BjfZtr3A6
ONUsbA+C1WowEih2iPcq2RqMyZx6w2srDS7hZBaMez/wO9qh4SBY/OJy9//m6aKKtY+HHXRYLZeGQDWlJ+b/Uf8Pni8ybTRX2
ZllnyWVWOTKA3QezdoJPZlA5syxeUtF2w8tBjN2KMGilcODHpqHax664LKvuku1K8/VP/rnSZFHz9r+Dz3o5K8l/hHmz0nAnW
zlEi04iaCFgb8sKfFHYQ1GKzmUZDzq1mxRQexKJxhxB2JjOER3ideFvDXvmU1sOOajTPjnsQD7iKekwVkOuJ8AIJ+zins/fOi
iYomHcSiyjSV8jUYU3WU1J8xnAM+gCxc6XSuK8/2YDw== kenzo@workstation
kenzo@workstation:~/ansible_proj$ ssh -T github.com
kenzo@github.com: Permission denied (publickey).
kenzo@workstation:~/ansible_proj$ ssh -T git@github.com
Hi qknlim! You've successfully authenticated, but GitHub does not provide shell access.
kenzo@workstation:~/ansible_proj$ git clone git@github.com:qknlim/CPE212_Lim
Cloning into 'CPE212_Lim'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 6 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (6/6), done.
kenzo@workstation:~/ansible_proj$ ls
ansible.cfg  CPE212_Lim  inventory.yaml  playbook.yaml
kenzo@workstation:~/ansible_proj$ cd CPE212_Lim
kenzo@workstation:~/ansible_proj/CPE212_Lim$ ls
README.md
kenzo@workstation:~/ansible_proj/CPE212_Lim$
```

(Copying ssh key into github and cloning because im using personal PC)



Run the yml file using the command: **ansible-playbook --ask-become-pass install_apache.yml.** Describe the result of this command.

```
kenzo@workstation:~/ansible_proj/CPE212_Lim$ ansible-playbook --ask-become-pass install_apache.ya
ml
BECOME password:

PLAY [all] ***********************************************************************************

TASK [Gathering Facts] ***********************************************************************
[WARNING]: Platform linux on host 192.168.56.106 is using the discovered Python interpreter at
/usr/bin/python3.9, but future installation of another Python interpreter could change the
meaning of that path. See https://docs.ansible.com/ansible-
core/2.17/reference_appendices/interpreter_discovery.html for more information.
ok: [192.168.56.106]

TASK [Install Apache2 Package] ***************************************************************
fatal: [192.168.56.106]: FAILED! => {"changed": false, "failures": ["No package apache2 available
."], "msg": "Failed to install some of the specified packages", "rc": 1, "results": []}

PLAY RECAP ***********************************************************************************
192.168.56.106             : ok=1    changed=0    unreachable=0    failed=1    skipped=0    rescu
ed=0    ignored=0
```

An error occurred because the name I put inside install_apache.yaml was apache2, where apache2 is a debian name and I searched about what apache is called in centos (httpd)

```
kenzo@workstation:~/ansible_proj/CPE212_Lim$ ansible-playbook --ask-become-pass install_apache.ya
ml
BECOME password:

PLAY [all] ***********************************************************************************

TASK [Gathering Facts] ***********************************************************************
[WARNING]: Platform linux on host 192.168.56.106 is using the discovered Python interpreter at
/usr/bin/python3.9, but future installation of another Python interpreter could change the
meaning of that path. See https://docs.ansible.com/ansible-
core/2.17/reference_appendices/interpreter_discovery.html for more information.
ok: [192.168.56.106]

TASK [Install Apache2 Package] ***************************************************************
changed: [192.168.56.106]

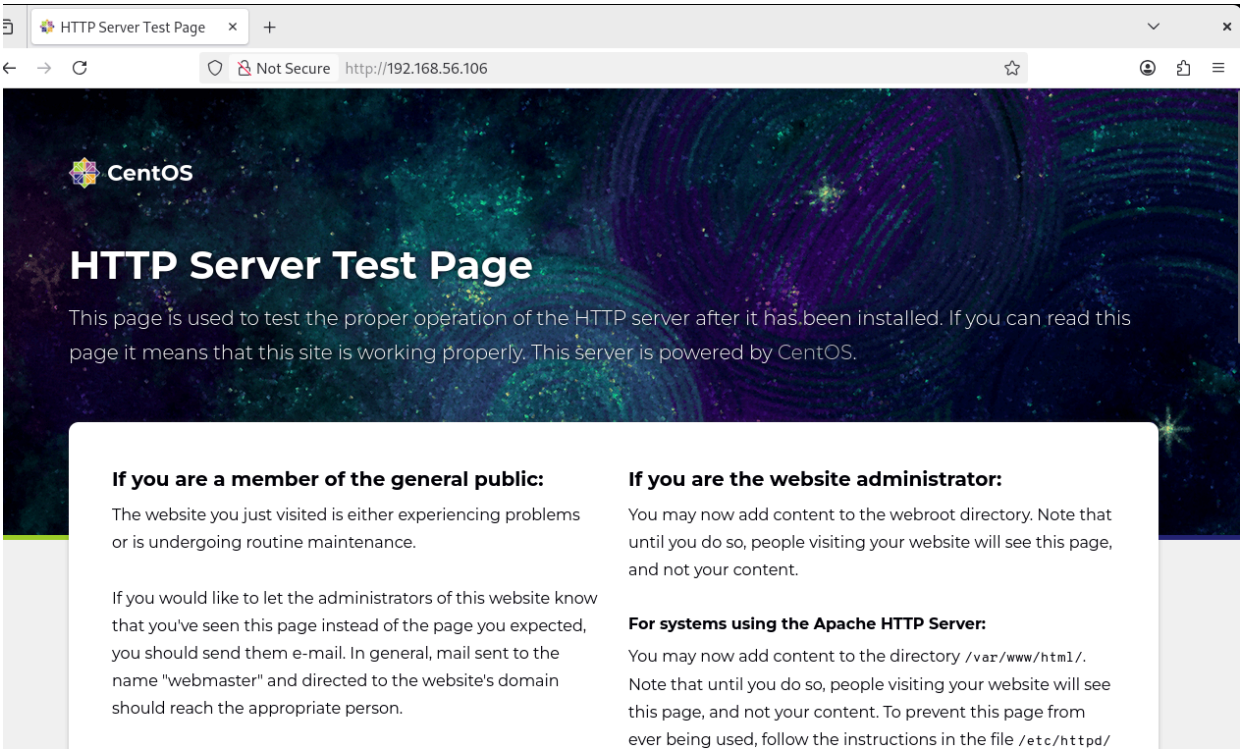
PLAY RECAP ***********************************************************************************
192.168.56.106             : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescu
ed=0    ignored=0
```

After changing the name to httpd and executing the install_apache.yaml, it was able output changed=1 in the TASK part

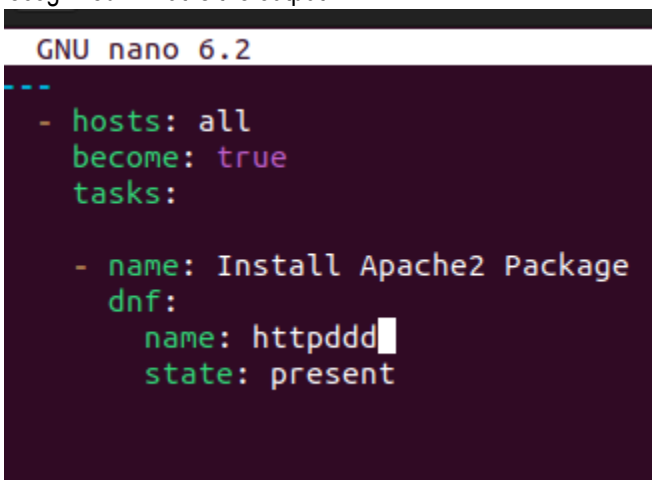
Verifying apache2

```
[kenzo@localhost log]$ sudo systemctl status httpd
○ httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
     Active: inactive (dead)
       Docs: man:httpd.service(8)
[kenzo@localhost log]$ sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd
/system/httpd.service.
[kenzo@localhost log]$ sudo systemctl status httpd
○ httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
     Active: inactive (dead)
       Docs: man:httpd.service(8)
[kenzo@localhost log]$
```

```
[kenzo@localhost log]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
     Active: active (running) since Mon 2026-08-24 14:21:52 PST; 3s ago
       Docs: man:httpd.service(8)
   Main PID: 41781 (httpd)
     Status: "Started, listening on: port 80"
      Tasks: 177 (limit: 10500)
     Memory: 19.0M (peak: 19.2M)
        CPU: 120ms
     CGroup: /system.slice/httpd.service
```



Try to edit the install_apache.yml and change the name of the package to any name that will not be recognized. What is the output?




```
kenzo@workstation:~/ansible_proj/CPE212_Lim$ ansible-playbook --ask-become-pass install_apache.ya
ml
BECOME password:

PLAY [all] *********************************************************************************

TASK [Gathering Facts] *********************************************************************
[WARNING]: Platform linux on host 192.168.56.106 is using the discovered Python interpreter at
/usr/bin/python3.9, but future installation of another Python interpreter could change the
meaning of that path. See https://docs.ansible.com/ansible-
core/2.17/reference_appendices/interpreter_discovery.html for more information.
ok: [192.168.56.106]

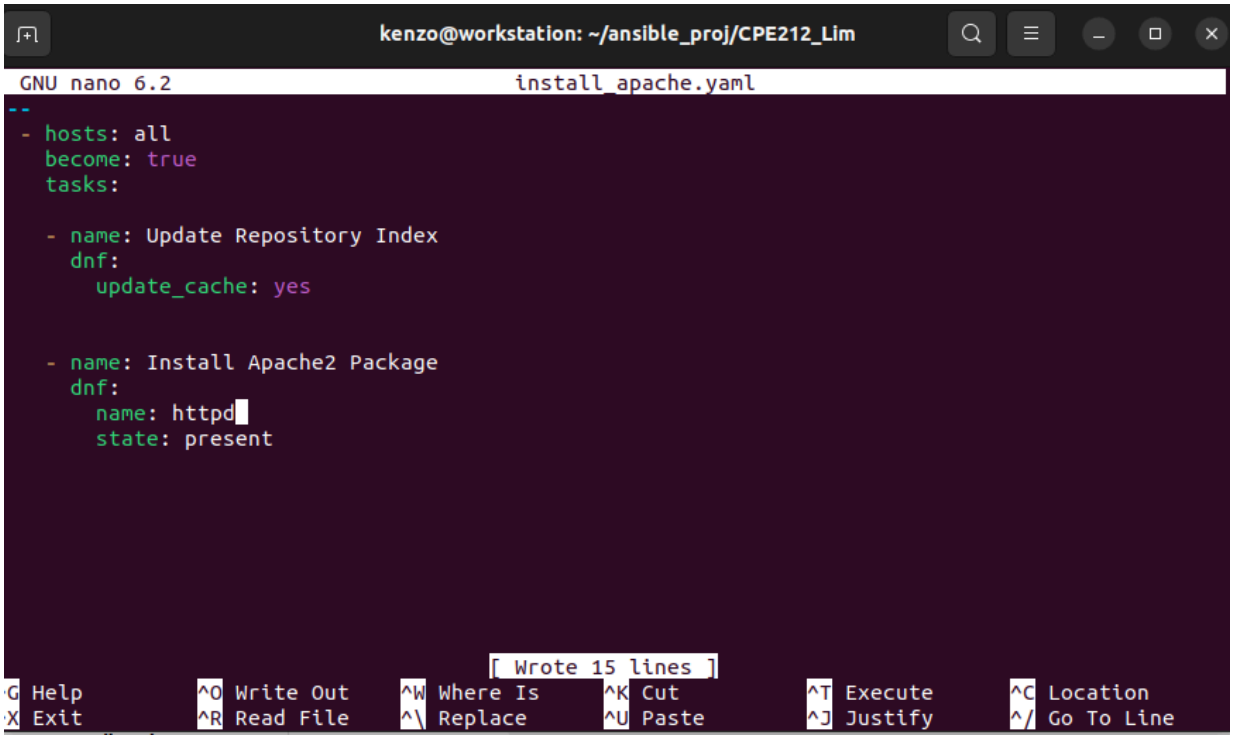
TASK [Install Apache2 Package] *************************************************************
fatal: [192.168.56.106]: FAILED! => {"changed": false, "failures": ["No package httpddd available
."], "msg": "Failed to install some of the specified packages", "rc": 1, "results": []}

PLAY RECAP *********************************************************************************
192.168.56.106             : ok=1    changed=0    unreachable=0    failed=1    skipped=0    rescu
ed=0    ignored=0

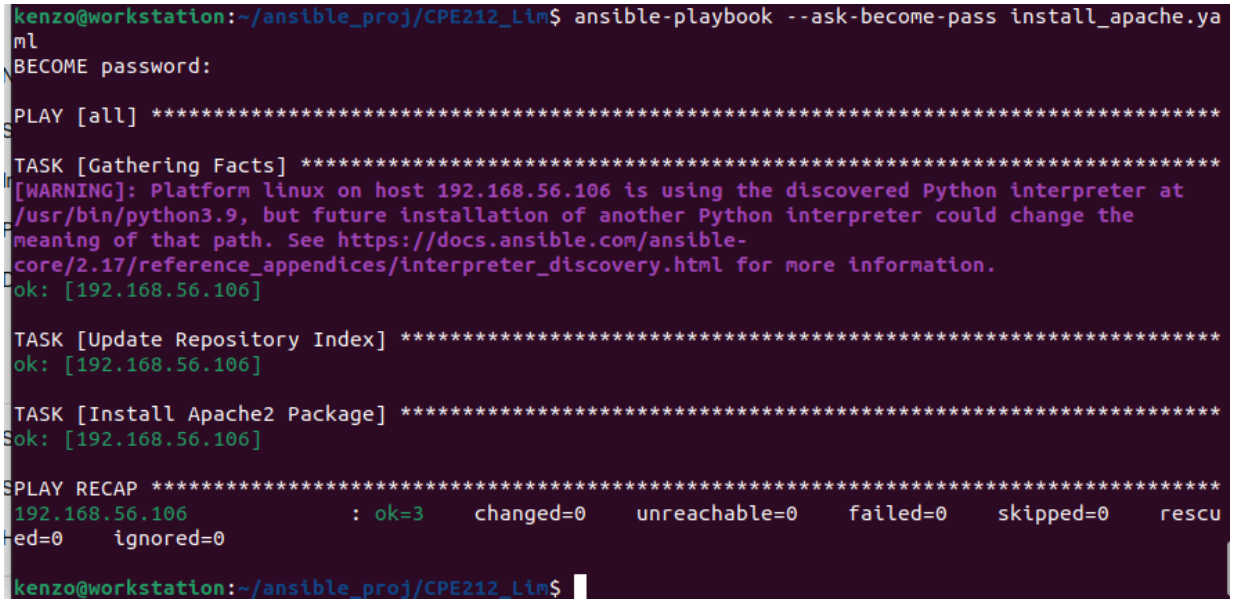
kenzo@workstation:~/ansible_proj/CPE212_Lim$
```

It failed as stated in error: "No package httpdddd available"

Edit the install_apache.yml. As you can see, we are now adding an additional command, which is the update_cache.



Run the playbook and describe the output. Did the new command change anything on the remote servers



It installed the httpd service in my CentOS server, however the state was inactive and hasn't started yet.

Edit again the install_apache.yml. This time, we are going to add a PHP support for the apache package we installed earlier

```
  GNU nano 6.2                                                in
---
  - hosts: all
    become: true
    tasks:

    - name: Update Repository Index
      dnf:
        update_cache: yes


    - name: Install Apache2 Package
      dnf:
        name: httpd
        state: present

    - name: Add PHP support
      dnf:
        name: php
```

```
PLAY [all] ********************************************************************************

TASK [Gathering Facts] ********************************************************************
[WARNING]: Platform linux on host 192.168.56.106 is using the discovered Python interpreter at
/usr/bin/python3.9, but future installation of another Python interpreter could change the
meaning of that path. See https://docs.ansible.com/ansible-
core/2.17/reference_appendices/interpreter_discovery.html for more information.
ok: [192.168.56.106]

TASK [Update Repository Index] ************************************************************
ok: [192.168.56.106]

TASK [Install Apache2 Package] ************************************************************
ok: [192.168.56.106]

TASK [Add PHP support] ********************************************************************
changed: [192.168.56.106]

PLAY RECAP ********************************************************************************
192.168.56.106             : ok=4    changed=1    unreachable=0    failed=0    skipped=0    rescu
ed=0    ignored=0
```

Run the playbook and describe the output. Did the new command change anything on the remote servers?

This time, apache2 now has support for php since we installed php service in my CentOS.

Finally, make sure that we are in sync with GitHub. Provide the link of your GitHub repository.

```
fatal: unable to auto-detect email address (got 'kenzo@workstation.(none)')
kenzo@workstation:~/ansible_proj/CPE212_Lim$ git config --global user.email "qknlim@tip.edu.ph"
kenzo@workstation:~/ansible_proj/CPE212_Lim$ git commit -m "HOA4"
[main dba0455] HOA4
 1 file changed, 19 insertions(+)
 create mode 100644 install_apache.yaml
kenzo@workstation:~/ansible_proj/CPE212_Lim$ git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 430 bytes | 430.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:qknlim/CPE212_Lim
   374ca0d..dba0455  main -> main
kenzo@workstation:~/ansible_proj/CPE212_Lim$
```

https://github.com/qknlim/CPE212_Lim

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Answer the following:

1. What is the importance of using a playbook?

Playbooks provide a more structured approach compared with running ad hoc commands, it allows tasks and variables, as well as configurations to be defined, which improves automation. Playbooks also provide error handling mechanisms that can control how failures are handled, it helps prevent unintended changes throughout the execution of the playbook

2. Summarize what we have done on this activity.

In this activity, we first started off with executing ad hoc commands by running apt modules (in my case I used dnf because my server was CentOS in my personal PC) to update the cache and install packages like vim-nox and snapd. Additionally, we automated using playbooks wherein we install apache2 on our servers, next is adding complexity which is the expansion of playbook to include repository updates and adding php support to our apache server, lastly pushing the latest version into github to update the repository.

Conclusions/Learnings:

To conclude, I was able to discover the meaning of ad hoc commands, and as well as execute them myself. From executing ad hoc commands into automating using playbook, I was able to familiarize myself with Ansible even more since this activity requires us to push the repository in github. Some challenges I encountered were basically package errors since I was using CentOS as my server, which I really had a trouble with since the instructions were debian/ubuntu based package, I had to look up for the same exact package counterpart debian/ubuntu has while im inside CentOS and modify the playbooks every instruction given. Overall, after realizing that this activity was Ubuntu-to-Ubuntu**(Client to Server)**, I was still able to accomplish the instructions while using Ubuntu-to-CentOS.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

COMPUTER ENGINEERING

QUEZON CITY

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Error Analysis**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I mainly utilized ChatGPT in handling errors. Additionally asking for the correct and exact opposite of Ubuntu / CentOS packages because this activity was supposed to be Ubuntu-to-Ubuntu (client and server), in my case I used Ubuntu-to-CentOS.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.